Lab 03 - Prompt Experiment Matrix for Controlled Image Generation

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 02: AI Image Generation, Editing and Visual Enhancement Outcome: ELO2 | In-class time: 15 minutes of scheduled practical time

Competency	Statement
Ability A2	Apply the principles of processing, filtering and analysis methods for video data
Knowledge K3	Methods to represent image and video data
Knowledge K4	Image and video processing, filtering and transformation methods

---

Scenario

A marketing team complains that 'the AI keeps changing the product'. Treat prompting as a designed experiment: compare progressively specified prompt bundles, generate or simulate a cell of the matrix, and score output variance numerically instead of arguing about it. You will end with a versioned JSON prompt that a colleague can reproduce. Classroom core (15 minutes): specify one constrained prompt and evaluate the supplied comparison set; generate one live result when account access permits. Repeated live trials and the full implementation walkthrough are extensions, not assumed complete in 15 minutes.

What you will produce

A completed factor-level table and four-run comparison, a versioned prompt-v3.json structured prompt, and out/variance-report.csv scoring the measured spread of each cell.

Tools: Text editor | Python 3 | opencv-python | NumPy | OPTIONAL free-tier image model. The lab completes fully offline using the supplied simulated variant sets.

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3, opencv-python, numpy.

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are *not* in that wheel and are deliberately avoided: cv2.TrackerCSRT_create / cv2.TrackerKCF_create and the cv2.quality SSIM module, both of which live in opencv-contrib-python. Where SSIM is needed the course ships its own NumPy implementation.

Workflow

1. Define the four prompt factors
2. Describe factor levels and the four run bundles
3. Generate or load each cell's variant set
4. Score within-cell variance numerically
5. Freeze your chosen prompt (based on brief fit, not variance alone) as versioned JSON

Files in this folder

Path	What it is
reference/variants/cellA_01.png .. cellD_04.png	16 supplied variant images - four cells of four variants each, deterministically rendered with OpenCV to reproduce the characteristic spread of an under-specified, partially specified, subject-locked and fully specified prompt. Every file carries a SIMULATED banner: these are classroom stand-ins for model output, NOT generative-model output.
data/prompt-factors.json	The four prompt factors and their levels, in the structure used by the scorer
data/campaign-brief.md	The NorthQuay 'Harbour Line' campaign brief the prompts must satisfy
data/prompt-v1.json	The under-specified starting prompt
data/prompt-v2.json	The partially specified prompt with subject lock
prompt_variance.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

Steps

1. Start inside this lab folder using the prepared Python/OpenCV/NumPy environment. Create out/ with your file manager. Record source filenames, model/settings if used and selected path in out/provenance.md. These simulations teach measurement only; their trend does not demonstrate a real model effect.

2. Read data/campaign-brief.md. Extract the four things the brief actually constrains: the SUBJECT (the product and who holds it), the CONTEXT (setting and time of day), the STYLE (visual register), and the TECHNICAL modifiers (lens, lighting, aspect ratio). These four are your experiment factors.

3. Open data/prompt-v1.json - the under-specified baseline. Note that it names only the subject. Predict, before measuring, which of the four factors will vary most between runs. Write your prediction down.

4. Build the experiment matrix. Rows are the four factors; columns are three levels: ABSENT, NAMED, LOCKED. 'Named' means the factor appears as a phrase; 'locked' means it appears as an explicit constrained value (for example "lens": "35mm" rather than 'wide shot'). Record the matrix in out/experiment-matrix.md.

5. Choose the four cells you will actually run: A = all factors absent except subject; B = subject named + context named; C = subject LOCKED + context named + style named; D = all four LOCKED. A full factorial design would have 3^4=81 combinations; these four bundles illustrate a comparison within the lab slot, not an isolated causal effect.

6. Write the four prompts. Record every change between bundles and explicitly note that several factors change together; causal attribution to one factor is not possible.

7. OPTIONAL - if you have free-tier access to an image model, generate four variants per cell with a recorded sampling policy (seed only if supported) and save them as out/gen/cellX_0N.png. If you do not, skip to the next step and use the supplied simulated set. Before live generation create out/prompt-A.json through out/prompt-D.json from the prompt pack; for prose cells use a JSON object with a prompt key. Either path completes the lab; record which one you used.

```
# Current documented Gemini image models (verified 6 Sep 2026):
#   gemini-3.1-flash-image, gemini-3.1-flash-lite-image, gemini-3-pro-image
#   (gemini-2.5-flash-image is the legacy model)
# All generated images carry a SynthID watermark.
# The imagen-4.0-* IDs used by older tutorials are deprecated.
# See PROMPTS.md for the exact request bodies.
```

8. For the offline route only, load the supplied variant sets from reference/variants/. Confirm there are 16 files in four cells of four, and that each carries the SIMULATED banner.

```
ls reference/variants | wc -l
```

9. Score within-cell variance on SUBJECT. For each cell, compute the mean pairwise Bhattacharyya distance between the HSV histograms of the subject region (the central 60% crop). Low distance means similar HS colour distributions in that crop, not proven subject identity or position.

```
python3 prompt_variance.py --variants reference/variants --out out/variance-report.csv
# For your generated files instead use:
# python3 prompt_variance.py --variants out/gen --out out/variance-report.csv
```

10. Exclude the banner from interpretation: the following metrics use the whole frame and its background. They are coarse brightness/layout proxies, not validated subject or style scores. Score within-cell variance on COMPOSITION. The scorer also reports the standard deviation of the subject centroid across each cell, in pixels. This is a limited proxy and cannot establish a prompt that keeps the product but moves it around the frame.

11. Score within-cell variance on STYLE. The scorer reports the standard deviation of edge density and of mean saturation across the cell - a proxy for 'is the visual register holding'.

12. Read out/variance-report.csv. For the designed simulated set inspect its intended trend: cell A has the highest histogram distance and centroid spread, and cell D the lowest. If your own generated set does not show that trend, record the result honestly; sampling uncertainty and model behaviour may explain it. Review the full change log without changing observations to fit expectations.

13. Compare the measured result against your step-2 prediction. Write one sentence assessing your prediction; state whether your prediction was supported; do not invent an error if it was correct.

14. Freeze your chosen prompt (based on brief fit, not variance alone) as out/prompt-v3.json. Use one key per factor so a single value can be changed without rewriting a sentence - that is the whole argument for structured prompting over prose.

15. Add the three enterprise fields to the frozen prompt: prompt_version, brief_reference and negative_constraints. A prompt with a version number is easier to audit alongside its hash and provenance.

16. Record the provenance expectation for anything you actually generated: which model, which prompt version, and the fact that current Gemini image outputs carry a SynthID watermark. If you used the simulated set, record that instead - never label a simulated asset as model output.

17. Save out/experiment-matrix.md, out/variance-report.csv and out/prompt-v3.json, then complete the evidence checklist.

Verify

out/variance-report.csv shows the mean pairwise histogram distance and centroid standard deviation reported for every cell A to D; real-model results need not be monotonic, and out/prompt-v3.json parses as valid JSON with one key per factor plus prompt_version, brief_reference and negative_constraints.

Expected outputs

- out/experiment-matrix.md - a 4x3 factor-by-level matrix with the four run cells marked.
- out/variance-report.csv - one row per cell with hist_distance_mean, centroid_std_px, edge_density_std and saturation_std.
- Report the observed trend and limitations; the simulated trend is designed, not evidence of prompting efficacy.
- out/prompt-v3.json - valid JSON, one key per factor, versioned.
- out/provenance.md with exact input filenames, settings, dimensions and limitations. A written statement of which generation path was used (free-tier model or supplied simulated set).

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

[ ] The 4x3 experiment matrix with the four run cells identified.

[ ] out/variance-report.csv pasted, showing the A-to-D trend.

[ ] Your step-2 prediction and the one-sentence correction after measuring.

[ ] out/prompt-v3.json including prompt_version and negative_constraints.

[ ] An explicit provenance line: model + prompt version + watermark expectation, OR a statement that the supplied simulated variant set was used.

Troubleshooting

Symptom	What to do
The variance trend is flat or reversed on your own generated images	This can be a valid stochastic result. Record it, inspect all bundle changes and model settings; an adjective such as 'cinematic' silently locks style in a cell that was supposed to leave it absent, but this is not the only possible cause.
prompt_variance.py reports 'need 4 variants per cell'	The one flat variants folder needs four files for each of cellA, cellB, cellC and cellD with the cell<X>_<NN>.png naming. Re-copy reference/variants/ from the lab folder.
Bhattacharyya distance is 0.0 for every pair	Different spatial arrangements or brightness can share the same HS histogram. Check for duplicates, and state that this feature does not verify identity or spatial structure.
A free-tier request is refused or rate-limited	Expected, and not a blocker. The lab is designed to complete on the supplied simulated set; record that path and move on. Never let a paid or rate-limited service gate a classroom deliverable.
You are tempted to submit a simulated variant as model output	Do not. Mislabelling provenance is an assessment failure and, in production, a governance breach. The SIMULATED banner is burned into the supplied images deliberately.

References used in this lab

- [S7] Tschochohei and Schenker (2026), as above - *Ch.5 pp.65-67*.

Prompt structure - subject, context, style, technical modifiers; increasing alignment with prompt length; subject lock and fixed photography style reduce output variance; structured JSON prompting for enterprise repeatability and versioning; repetition and contradictory instructions degrade output.

- [S12] Tschochohei and Schenker (2026), as above - *Ch.8 pp.127-132 and pp.140-143*.

Modality-specific responsible-AI risks (stereotypes and representational harm, training-data and output ownership, deceptive imagery, voice cloning, deepfakes); provider safety comparison (SynthID watermarking and model cards; C2PA Content Credentials metadata; banned-word filters; configurable safety tolerance); transparency strategy - C2PA content credentials, robust invisible watermarking resilient to compression/cropping/format change, and model cards as a 'nutrition label for AI'.

- [S17] Mewada et al. (2026), as above - *Ch.7 pp.94-98 (Dubey, Pinheiro, Singh, Ansari and Kumar, 'Advanced Foundations and Future Trends in Generative AI for Visual Media')*.

Table 7.2 scaling strategies and their targets (trajectory distillation -> fewer sampling steps at similar FID/FVD; quantisation 8/4-bit -> reduced latency and memory; low-rank adapters -> small trainable-parameter fraction; latent diffusion -> lower bandwidth cost at high resolution; retrieval conditioning -> higher faithfulness without more parameters). Evaluation protocol: FID as a 2-Wasserstein approximation in feature space (encoder-dependent, sample-size sensitive), KID as an unbiased MMD^2, LPIPS as a learned perceptual distance, precision/recall for mode coverage, CLIP-based text-image faithfulness, masked-region consistency and identity preservation for edits, FVD plus optical-flow agreement for video, and watermark detectability under common edits (Table 7.3 axes and pitfalls).

- [S20] Google AI for Developers - Gemini API image generation documentation - *https://ai.google.dev/gemini-api/docs/image-generation (verified 6 Sep 2026)*.

Current image-generation model IDs gemini-3.1-flash-image, gemini-3.1-flash-lite-image, gemini-3-pro-image, with gemini-2.5-flash-image as the legacy model; the client.interactions.create call pattern for text-to-image, image editing and multi-turn conversational editing via previous_interaction_id; aspect ratios including 1:1, 3:2, 2:3, 3:4, 4:3, 4:5, 5:4, 9:16, 16:9 and 21:9; output sizes 1K/2K/4K by model; and the statement that all generated images include a SynthID watermark.

- [S21] Google AI for Developers - Gemini API Imagen documentation - *https://ai.google.dev/gemini-api/docs/imagen (verified 6 Sep 2026)*.

CURRENCY CORRECTION: the imagen-4.0-generate-001 family used in the reference ebook is documented as deprecated with shutdown on 17 August 2026, and the Gemini native image models are the documented replacement. Course materials therefore teach the current Gemini image model IDs and treat the ebook's Imagen code as historical.
